🔬 PHÂN TÍCH SIÊU CHI TIẾT: Cách Tạo Extension VEO Automation

> ***VEO Automation - Auto VEO on Google Flow*** *| Dev:* ***Truong Nguyen*** *(kylenguyen.me) | vn Hà Nội*
> *v2.3.6 | 711 KiB | 10,000+ users |* ⭐ *4.7/5 (31 reviews)*

📸 Giao Diện Thực Tế Của Extension

```
![Screenshot 1 - Side Panel chính: Tabs Control/Setting, các mode Text to Video,
Frame to Video, Ingredients to Video, Text to Image, Image to Image, Last Image To
Image. Phần Video Generation Progress hiển thị thanh tiến trình cho từng prompt]
(C:\Users\intel\.gemini\antigravity\brain\830c6321-dd06-4812-adc3-
344869cea7c5\veo_ss2_1772203898902.png)
<!-- slide -->
![Screenshot 2 - Queue Management: Danh sách prompts với thumbnail ảnh, Duration per
Prompt (8s), progress tracking, character count cho mỗi item]
(C:\Users\intel\.gemini\antigravity\brain\830c6321-dd06-4812-adc3-
344869cea7c5\veo_ss3_1772203931649.png)
```

I. 👩‍💻 DEVELOPER: Hệ Sinh Thái Sản Phẩm

> *[!IMPORTANT]* ***Truong Nguyen*** *không chỉ tạo 1 extension — anh ấy xây dựng* ***hệ sinh thái 4 extensions*** *cùng kiến trúc, chứng tỏ đây là một* ***template có thể tái sử dụng****:*

#	Extension	Target Platform	Link
1	**VEO Automation**	Google Flow (VEO)	Install
2	**Grok Automation**	grok.com	Install
3	**Whisk Automation**	Google Whisk (Banana)	Install
4	**Meta AI Automation**	meta.ai	Install

Insight kỹ thuật: Cả 4 extensions đều dùng chung **core architecture**: Side Panel UI + Service Worker Queue + Content Script DOM Automation. Chỉ cần thay đổi **content script selectors** cho từng platform.

II. 🏗️ KIẾN TRÚC TỔNG THỂ — Chrome Extension Manifest V3

```
graph TB
    subgraph "🧩 Chrome Extension Package (711 KiB)"
        direction TB
        MF["📄 manifest.json\n(Manifest V3)"]

        subgraph "Layer 1: UI"
            SP["🖥️ Side Panel\npanel.html + panel.js + panel.css\n• Tab Control / Setting\n• Queue list + Progress bars\n• Language selector (6 langs)\n• Mode buttons (6 modes)"]
        end

        subgraph "Layer 2: Orchestration"
            SW["⚙️ Service Worker\nservice-worker.js\n• Queue Manager (FIFO + concurrency)\n• Job scheduler\n• Download coordinator\n• Message hub"]
        end

        subgraph "Layer 3: Automation"
            CS["🚀 Content Script\nflow-automator.js\n• DOM traversal & manipulation\n• React input hacking\n• MutationObserver polling\n• Fetch/XHR interception"]
            MAIN["🔓 MAIN World Script\npage-interceptor.js\n• Override window.fetch\n• Capture API responses\n• Extract video URLs"]
        end

        subgraph "Layer 4: Data"
            STORE["💾 chrome.storage.local\n• Queue state persistence\n• User settings\n• Language preference"]
            I18N["🌐 _locales/\n• en, vi, zh, ko, ja, es"]
        end
    end

    subgraph "🌐 Google Flow Website"
        GF_UI["Google Flow UI\n(React SPA)"]
        GF_API["aisandbox-pa.googleapis.com\n• POST /generate\n• GET /status\n• GET /download"]
    end

    SP <-->|"chrome.runtime.sendMessage()"| SW
    SW <-->|"chrome.tabs.sendMessage()"| CS
    CS -->|"Inject vào MAIN world"| MAIN
    CS <-->|"DOM read/write\nquerySelector, click,\ndispatchEvent"| GF_UI
```

```
    MAIN -->|"Intercept fetch()\n→ capture video URL"| GF_API
    GF_UI <-->|"Internal React\nfetch() calls"| GF_API
    SW -->|"chrome.downloads.download()"| DL["📥 User's Downloads"]
    SW <-->|"get/set"| STORE

    style SP fill:#1a1a2e,stroke:#e94560,color:#fff
    style SW fill:#16213e,stroke:#0f3460,color:#fff
    style CS fill:#0f3460,stroke:#533483,color:#fff
    style MAIN fill:#533483,stroke:#e94560,color:#fff
```

III. 📄 manifest.json — Giải Thích TỪNG DÒNG

```
{
  // ===== PHIÊN BẢN MANIFEST =====
  "manifest_version": 3,          // BẮT BUỘC dùng V3 (V2 đã bị Chrome deprecate)
                                   // V3 thay background page → service worker
                                   // V3 không cho load remote code từ CDN

  // ===== THÔNG TIN CƠ BẢN =====
  "name": "__MSG_extensionName__", // Dùng i18n key → hiển thị tên theo ngôn ngữ
browser
  "version": "2.3.6",             // SemVer: Major.Minor.Patch
  "description": "__MSG_extensionDescription__",
  "default_locale": "en",         // Ngôn ngữ mặc định nếu browser không match

  // ===== PERMISSIONS (Quyền truy cập) =====
  "permissions": [
    "sidePanel",    // Mở Side Panel UI persistent bên cạnh webpage
    "storage",      // chrome.storage.local — lưu queue, settings (KHÔNG phải
cookies!)
    "downloads",    // chrome.downloads.download() — tải video/ảnh tự động
    "scripting",    // chrome.scripting.executeScript() — inject script động vào tab
    "tabs",         // chrome.tabs.query/sendMessage — tìm tab và gửi lệnh
    "activeTab"     // Truy cập tab đang active khi user click extension icon
  ],

  // ===== HOST PERMISSIONS (Website được phép thao tác) =====
  "host_permissions": [
    "https://labs.google.com/fx/tools/flow/*",      // Google Flow UI
    "https://aisandbox-pa.googleapis.com/*"          // Google Flow backend API
```

```
    // ↑ Extension CHỈ chạy trên 2 domain này = bảo mật tốt
  ],

  // ===== BACKGROUND SERVICE WORKER =====
  "background": {
    "service_worker": "background/service-worker.js",
    "type": "module"    // Cho phép dùng ES import/export
    // QUAN TRỌNG: Service Worker KHÔNG có DOM!
    // Không thể dùng document.querySelector() ở đây
    // Nó là "bộ não" điều phối, không phải "tay chân"
  },

  // ===== CONTENT SCRIPTS (Inject tự động vào webpage) =====
  "content_scripts": [{
    "matches": ["https://labs.google.com/fx/tools/flow/*"],
    "js": [
      "content-scripts/flow-automator.js"
      // File này chạy TRONG context của Google Flow
      // Nó có thể document.querySelector() trên Google Flow DOM
    ],
    "run_at": "document_idle"
    // document_idle = chạy SAU KHI DOM đã load xong
    // Alternatives: document_start (trước DOM), document_end (sau DOM, trước
resources)
  }],

  // ===== SIDE PANEL =====
  "side_panel": {
    "default_path": "side-panel/panel.html"
    // HTML file hiển thị trong Chrome Side Panel
    // Panel này PERSISTENT — không bị đóng khi switch tab!
  },

  // ===== EXTENSION ICON =====
  "action": {
    "default_icon": "icons/icon-32.png",  // Icon trên toolbar Chrome
    "default_title": "VEO Automation"     // Tooltip khi hover
  },

  "icons": {
    "16": "icons/icon-16.png",    // Favicon
    "48": "icons/icon-48.png",    // Extension management page
    "128": "icons/icon-128.png"   // Chrome Web Store listing (con hamster)
  }
}
```

IV. ⚙️ SERVICE WORKER — Bộ Não Điều Phối (Chi Tiết Từng Dòng)

```
// ==========================================================
// background/service-worker.js
// KHÔNG CÓ DOM! Không dùng document/window.
// Đây là event-driven worker, chỉ chạy khi có event.
// ==========================================================

// ─── KHỞI TẠO KHI CÀI ĐẶT EXTENSION ───
chrome.runtime.onInstalled.addListener(({ reason }) => {
  if (reason === 'install') {
    // Lần đầu cài: set default settings
    chrome.storage.local.set({
      settings: {
        maxConcurrent: 2,        // Tối đa 2 job song song
        autoDownload: true,      // Tự động tải khi xong
        downloadQuality: '1080p',// Chất lượng mặc định
        autoChaining: true,      // Tự link last frame → next prompt
        language: 'en'
      },
      queue: [],
      history: []
    });
  }
  // Kích hoạt Side Panel khi click icon
  chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick: true });
});

// ─── QUEUE MANAGER CLASS ───
class QueueManager {
  constructor() {
    this.queue = [];          // Jobs chờ xử lý
    this.running = new Map(); // Jobs đang chạy: jobId → metadata
    this.completed = [];      // Jobs đã hoàn thành
    this.failed = [];         // Jobs bị lỗi
    this.maxConcurrent = 2;
    this.isPaused = false;
  }

  // Thêm hàng loạt jobs vào queue
```

```
  async addBatch(promptTexts, config) {
    const jobs = promptTexts.map((prompt, index) => ({
      id: `job_${Date.now()}_${index}`,
      prompt: prompt.trim(),
      charCount: prompt.trim().length,    // Hiển thị trên UI (như screenshot)
      mode: config.mode,                   // 'text-to-video' | 'image-to-video' |
...
      model: config.model,                 // 'veo-3.1-fast' | 'veo-3.1-quality' |
...
      aspectRatio: config.aspectRatio,     // '16:9' | '9:16' | '1:1'
      duration: config.duration,           // '5s' | '8s'
      referenceImage: null,                // Sẽ được set nếu autoChaining=true
      status: 'pending',                   // pending → running → completed/failed
      progress: 0,                         // 0-100%
      createdAt: Date.now(),
      videoUrl: null,
      thumbnailUrl: null,
      error: null
    }));

    this.queue.push(...jobs);
    await this.persistState();
    this.broadcastUpdate();
    this.processNext();
    return jobs.length;
  }

  async processNext() {
    if (this.isPaused) return;
    if (this.running.size >= this.maxConcurrent) return;
    if (this.queue.length === 0) return;

    const job = this.queue.shift();
    job.status = 'running';
    this.running.set(job.id, job);
    this.broadcastUpdate();

    try {
      // ═══ BƯỚC QUAN TRỌNG: Auto-Chaining ═══
      // Nếu bật autoChaining VÀ có job trước đã hoàn thành
      // → lấy last frame của video trước làm reference image cho job này
      if (this.completed.length > 0) {
        const lastCompleted = this.completed[this.completed.length - 1];
        if (lastCompleted.lastFrameUrl) {
          job.referenceImage = lastCompleted.lastFrameUrl;
        }
```

```
      }

      // Tìm tab Google Flow
      const tabs = await chrome.tabs.query({
        url: 'https://labs.google.com/fx/tools/flow/*'
      });

      if (tabs.length === 0) {
        throw new Error('Google Flow tab not found. Please open Google Flow first.');
      }

      // ═══ GỬI LỆNH XUỐNG CONTENT SCRIPT ═══
      const result = await chrome.tabs.sendMessage(tabs[0].id, {
        action: 'EXECUTE_AUTOMATION',
        payload: job
      });

      if (!result.success) throw new Error(result.error);

      // Cập nhật kết quả
      job.status = 'completed';
      job.progress = 100;
      job.videoUrl = result.videoUrl;
      job.lastFrameUrl = result.lastFrameUrl;
      this.completed.push(job);

      // ═══ TỰ ĐỘNG TẢI VIDEO ═══
      const settings = await this.getSettings();
      if (settings.autoDownload && job.videoUrl) {
        await this.downloadResult(job);
      }

    } catch (err) {
      job.status = 'failed';
      job.error = err.message;
      this.failed.push(job);
    } finally {
      this.running.delete(job.id);
      await this.persistState();
      this.broadcastUpdate();
      // ═══ TIẾP TỤC JOB TIẾP THEO ═══
      this.processNext();
    }
  }

  async downloadResult(job) {
```

```
    const filename = this.sanitizeFilename(job.prompt).slice(0, 50);
    await chrome.downloads.download({
      url: job.videoUrl,
      filename: `VEO_Output/${job.id}_${filename}.mp4`,
      conflictAction: 'uniquify'  // Thêm (1), (2)... nếu trùng
    });
  }

  // Broadcast trạng thái lên Side Panel UI (realtime update)
  broadcastUpdate() {
    chrome.runtime.sendMessage({
      event: 'QUEUE_UPDATE',
      data: {
        queue: this.queue,
        running: [...this.running.values()],
        completed: this.completed,
        failed: this.failed,
        stats: {
          total: this.queue.length + this.running.size + this.completed.length,
          pending: this.queue.length,
          running: this.running.size,
          done: this.completed.length,
          errors: this.failed.length
        }
      }
    });
  }

  // Lưu vào chrome.storage để survive browser restart / SW termination
  async persistState() {
    await chrome.storage.local.set({
      queue: this.queue,
      running: [...this.running.values()],
      completed: this.completed,
      failed: this.failed
    });
  }

  // Khôi phục state sau khi SW bị terminate rồi restart
  async restoreState() {
    const data = await chrome.storage.local.get(['queue','completed','failed']);
    this.queue = data.queue || [];
    this.completed = data.completed || [];
    this.failed = data.failed || [];
    // Resume processing nếu còn jobs pending
    if (this.queue.length > 0) this.processNext();
```

```
  }

  sanitizeFilename(str) {
    return str.replace(/[<>:"/\\|?*]/g, '_').replace(/\s+/g, '_');
  }

  async getSettings() {
    const { settings } = await chrome.storage.local.get('settings');
    return settings;
  }
}

const queueManager = new QueueManager();
queueManager.restoreState(); // Khôi phục khi SW khởi động

// ─── MESSAGE ROUTER: Nhận lệnh từ Side Panel & Content Script ───
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  switch (msg.action) {
    case 'ADD_JOBS':
      queueManager.addBatch(msg.prompts, msg.config)
        .then(count => sendResponse({ status: 'ok', count }));
      return true; // async response

    case 'PAUSE_QUEUE':
      queueManager.isPaused = true;
      sendResponse({ paused: true });
      break;

    case 'RESUME_QUEUE':
      queueManager.isPaused = false;
      queueManager.processNext();
      sendResponse({ paused: false });
      break;

    case 'GET_STATUS':
      queueManager.broadcastUpdate();
      break;

    case 'PROGRESS_UPDATE': // Nhận từ Content Script
      const job = queueManager.running.get(msg.jobId);
      if (job) {
        job.progress = msg.progress;
        queueManager.broadcastUpdate();
      }
      break;
```

```
  }
});
```

V. 💉 CONTENT SCRIPT — DOM Automation Chuyên Sâu

> *[!CAUTION] Đây là phần **KHÓ NHẤT VÀ PHỨC TẠP NHẤT** của extension. Google Flow là React SPA — mọi DOM manipulation đều phải "hack" qua React.*

```
// ========================================================
// content-scripts/flow-automator.js
// Chạy TRONG context trang labs.google.com/fx/tools/flow
// Có access DOM của Google Flow nhưng KHÔNG access window JS của page
// (chạy trong Isolated World)
// ========================================================

// ═══════════════════════════════════════════════
// PHẦN 1: UTILITIES — Các hàm helper cốt lõi
// ═══════════════════════════════════════════════

/**
 * Chờ element xuất hiện trong DOM (cho React async rendering)
 * Google Flow render async — element không có ngay khi page load
 * Dùng MutationObserver thay vì setTimeout (đáng tin cậy hơn)
 */
function waitForElement(selector, timeout = 15000) {
  return new Promise((resolve, reject) => {
    // Check ngay — có thể element đã tồn tại
    const existing = document.querySelector(selector);
    if (existing) return resolve(existing);

    const observer = new MutationObserver((mutations, obs) => {
      const el = document.querySelector(selector);
      if (el) {
        obs.disconnect();
        resolve(el);
      }
    });

    observer.observe(document.body, {
      childList: true,   // Phát hiện thêm/xóa child nodes
```

```
      subtree: true,     // Theo dõi TOÀN BỘ DOM tree, không chỉ children trực tiếp
      attributes: true   // Phát hiện thay đổi attributes (class, style...)
    });

    setTimeout(() => {
      observer.disconnect();
      reject(new Error(`Element "${selector}" not found after ${timeout}ms`));
    }, timeout);
  });
}

/**
 * Chờ element BIẾN MẤT (ví dụ: loading spinner)
 */
function waitForElementGone(selector, timeout = 120000) {
  return new Promise((resolve, reject) => {
    const check = () => {
      if (!document.querySelector(selector)) return resolve();
      requestAnimationFrame(check);
    };
    check();
    setTimeout(() => reject(new Error('Element still exists: ' + selector)),
timeout);
  });
}

/**
 * ★★★ TRICK QUAN TRỌNG NHẤT ★★★
 * Set giá trị vào React-controlled input
 *
 * Tại sao input.value = "text" KHÔNG HOẠT ĐỘNG?
 * → React dùng "controlled components" — nó override native setter
 * → React chỉ update state khi nhận được synthetic event
 * → Ta phải gọi NATIVE setter rồi dispatch event
 */
function setReactInputValue(element, value) {
  // Xác định prototype dựa trên loại element
  const proto = element instanceof HTMLTextAreaElement
    ? HTMLTextAreaElement.prototype
    : HTMLInputElement.prototype;

  // Lấy native setter (bỏ qua React override)
  const nativeSetter = Object.getOwnPropertyDescriptor(proto, 'value').set;

  // Set giá trị qua native setter
  nativeSetter.call(element, value);
```

```
  // Dispatch events mà React lắng nghe
  element.dispatchEvent(new Event('input', { bubbles: true, cancelable: true }));
  element.dispatchEvent(new Event('change', { bubbles: true, cancelable: true }));

  // Một số React versions cần thêm KeyboardEvent
  element.dispatchEvent(new KeyboardEvent('keydown', { bubbles: true }));
  element.dispatchEvent(new KeyboardEvent('keyup', { bubbles: true }));
}

/**
 * Click element theo cách React nhận ra
 * Đôi khi element.click() không đủ cho React
 */
function reactClick(element) {
  // Dispatch full sequence: mousedown → mouseup → click
  element.dispatchEvent(new MouseEvent('mousedown', { bubbles: true }));
  element.dispatchEvent(new MouseEvent('mouseup', { bubbles: true }));
  element.dispatchEvent(new MouseEvent('click', { bubbles: true }));
}

function sleep(ms) {
  return new Promise(r => setTimeout(r, ms));
}

// ═══════════════════════════════════════════
// PHẦN 2: SELECTOR STRATEGY — Tìm element đúng
// ═══════════════════════════════════════════

/**
 * Google Flow thường xuyên thay đổi class names (hashed CSS modules)
 * VÍ DỤ: class="css-1a2b3c" → thay đổi mỗi lần deploy
 *
 * CHIẾN LƯỢC: Dùng multiple fallback selectors
 * Ưu tiên: aria-label > data-* > role > text content > class
 */
const SELECTORS = {
  // Nút tạo project mới
  newProject: [
    '[aria-label*="New"]',
    '[data-action="new-flow"]',
    'button:has(svg[data-icon="plus"])',
    // Fallback: tìm bằng text content
    () => [...document.querySelectorAll('button')]
      .find(b => b.textContent.includes('New'))
  ],
```

```
  // Textarea nhập prompt
  promptInput: [
    'textarea[placeholder*="prompt"]',
    'textarea[placeholder*="Describe"]',
    'textarea[aria-label*="prompt"]',
    '.prompt-input textarea',
    'textarea'  // Last resort
  ],

  // Nút Generate
  generateButton: [
    'button[aria-label*="Generate"]',
    'button[aria-label*="Create"]',
    '[data-action="generate"]',
    () => [...document.querySelectorAll('button')]
      .find(b => b.textContent.match(/Generate|Create|Submit/i))
  ],

  // Model selector dropdown
  modelSelector: [
    '[aria-label*="model"]',
    '[data-testid="model-selector"]',
    'select[name*="model"]'
  ],

  // Aspect ratio buttons
  aspectRatio: {
    '16:9': '[aria-label*="16:9"], [data-ratio="16:9"]',
    '9:16': '[aria-label*="9:16"], [data-ratio="9:16"]',
    '1:1':  '[aria-label*="1:1"], [data-ratio="1:1"]'
  },

  // Progress/completion indicators
  progressBar: '[role="progressbar"], [aria-valuenow]',
  videoElement: 'video[src], video source[src]',
  completionBadge: '[data-status="complete"], .generation-complete',
  errorIndicator: '[data-status="error"], .generation-error'
};

/**
 * Tìm element từ danh sách selectors (fallback chain)
 */
async function findElement(selectorList) {
  if (!Array.isArray(selectorList)) selectorList = [selectorList];
```

```
  for (const sel of selectorList) {
    try {
      if (typeof sel === 'function') {
        const el = sel();
        if (el) return el;
      } else {
        const el = await waitForElement(sel, 3000);
        if (el) return el;
      }
    } catch { /* try next */ }
  }
  throw new Error('No element found for selectors: ' + JSON.stringify(selectorList));
}

// ═══════════════════════════════════════════
// PHẦN 3: MAIN AUTOMATION FLOW
// ═══════════════════════════════════════════

chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  if (msg.action === 'EXECUTE_AUTOMATION') {
    executeAutomation(msg.payload)
      .then(result => sendResponse(result))
      .catch(err => sendResponse({ success: false, error: err.message }));
    return true; // Keep message channel open for async
  }
});

async function executeAutomation(job) {
  // ────── STEP 1: Tạo Project mới ──────
  const newBtn = await findElement(SELECTORS.newProject);
  reactClick(newBtn);
  await sleep(2000); // Chờ animation transition

  // ────── STEP 2: Chọn Generation Mode ──────
  await selectMode(job.mode);
  await sleep(500);

  // ────── STEP 3: Upload reference image (nếu có) ──────
  if (job.referenceImage && (job.mode === 'image-to-video' || job.mode === 'image-to-
image')) {
    await uploadReferenceImage(job.referenceImage);
    await sleep(1000);
  }

  // ────── STEP 4: Điền Prompt ──────
  const textarea = await findElement(SELECTORS.promptInput);
```

```
  textarea.focus();
  await sleep(200);
  setReactInputValue(textarea, job.prompt);
  await sleep(500);

  // Report progress
  chrome.runtime.sendMessage({ action: 'PROGRESS_UPDATE', jobId: job.id, progress: 20
});

  // ═══ STEP 5: Chọn Model ═══
  await selectModel(job.model); // 'veo-3.1-fast', 'veo-3.1-quality', etc.
  await sleep(300);

  // ═══ STEP 6: Chọn Aspect Ratio ═══
  const ratioBtn = await waitForElement(SELECTORS.aspectRatio[job.aspectRatio],
5000);
  reactClick(ratioBtn);
  await sleep(300);

  // ═══ STEP 7: Set Duration ═══
  await selectDuration(job.duration);
  await sleep(300);

  chrome.runtime.sendMessage({ action: 'PROGRESS_UPDATE', jobId: job.id, progress: 30
});

  // ═══ STEP 8: CLICK GENERATE! ═══
  const genBtn = await findElement(SELECTORS.generateButton);
  reactClick(genBtn);

  chrome.runtime.sendMessage({ action: 'PROGRESS_UPDATE', jobId: job.id, progress: 40
});

  // ═══ STEP 9: POLLING — Chờ video hoàn thành ═══
  const result = await pollForCompletion(job);

  return result;
}

/**
 * Poll liên tục để check video đã render xong chưa
 * Google Flow mất 30s-5 phút tùy model
 */
async function pollForCompletion(job) {
  const startTime = Date.now();
  const MAX_WAIT = 10 * 60 * 1000; // Timeout 10 phút
```

```
  while (Date.now() - startTime < MAX_WAIT) {
    // Check lỗi
    const error = document.querySelector(SELECTORS.errorIndicator);
    if (error) throw new Error('Generation failed: ' + (error.textContent || 'Unknown
error'));

    // Check progress bar
    const progressBar = document.querySelector(SELECTORS.progressBar);
    if (progressBar) {
      const value = parseInt(progressBar.getAttribute('aria-valuenow') || '0');
      chrome.runtime.sendMessage({
        action: 'PROGRESS_UPDATE', jobId: job.id,
        progress: Math.min(40 + value * 0.5, 90) // Map 0-100 → 40-90%
      });
    }

    // Check video element xuất hiện
    const video = document.querySelector(SELECTORS.videoElement);
    const complete = document.querySelector(SELECTORS.completionBadge);

    if (video || complete) {
      chrome.runtime.sendMessage({ action: 'PROGRESS_UPDATE', jobId: job.id,
progress: 95 });

      // Extract video URL
      const videoUrl = await extractVideoUrl();
      const lastFrameUrl = await captureLastFrame(video);

      return {
        success: true,
        videoUrl: videoUrl,
        lastFrameUrl: lastFrameUrl // Cho auto-chaining
      };
    }

    await sleep(3000); // Poll mỗi 3 giây
  }

  throw new Error('Generation timed out after 10 minutes');
}
```

VI. 🔓 MAIN WORLD INJECTION — Intercept Google Flow API

> *[!WARNING] Content Script chạy trong **Isolated World** — KHÔNG thể đọc biến JS của page.*
> *Để intercept `fetch()` calls của Google Flow → phải inject script vào **MAIN World**.*

```
// ═══════════════════════════════════════════
// Inject vào MAIN world từ content script
// ═══════════════════════════════════════════

function injectPageScript() {
  const script = document.createElement('script');
  script.textContent = `
    (function() {
      // ─── Override window.fetch ───
      const originalFetch = window.fetch;
      window.fetch = async function(...args) {
        const response = await originalFetch.apply(this, args);
        const url = typeof args[0] === 'string' ? args[0] : args[0]?.url;

        // Intercept video download URLs từ aisandbox-pa.googleapis.com
        if (url && url.includes('aisandbox-pa.googleapis.com')) {
          try {
            const clone = response.clone();
            const data = await clone.json();

            // Tìm video URL trong response
            if (data.downloadUrl || data.videoUri || data.mediaUrl) {
              // Gửi URL về Content Script qua CustomEvent
              window.dispatchEvent(new CustomEvent('__VEO_VIDEO_URL__', {
                detail: {
                  url: data.downloadUrl || data.videoUri || data.mediaUrl,
                  quality: data.quality || 'unknown',
                  type: data.mediaType || 'video'
                }
              }));
            }

            // Tìm generation status
            if (data.state || data.status) {
              window.dispatchEvent(new CustomEvent('__VEO_STATUS__', {
                detail: { state: data.state || data.status, progress: data.progress }
              }));
            }
```

```
          } catch(e) { /* không phải JSON, bỏ qua */ }
        }
        return response;
      };

      // ─── Override XMLHttpRequest (backup) ───
      const origOpen = XMLHttpRequest.prototype.open;
      XMLHttpRequest.prototype.open = function(method, url, ...rest) {
        this._interceptedUrl = url;
        return origOpen.apply(this, [method, url, ...rest]);
      };

      const origSend = XMLHttpRequest.prototype.send;
      XMLHttpRequest.prototype.send = function(...args) {
        this.addEventListener('load', function() {
          if (this._interceptedUrl?.includes('aisandbox-pa.googleapis.com')) {
            try {
              const data = JSON.parse(this.responseText);
              if (data.downloadUrl) {
                window.dispatchEvent(new CustomEvent('__VEO_VIDEO_URL__', {
                  detail: { url: data.downloadUrl }
                }));
              }
            } catch(e) {}
          }
        });
        return origSend.apply(this, args);
      };
    })();
  `;
  document.documentElement.appendChild(script);
  script.remove(); // Xóa tag <script> sau khi đã execute
}

// Content Script lắng nghe CustomEvent từ MAIN world
let capturedVideoUrl = null;
window.addEventListener('__VEO_VIDEO_URL__', (e) => {
  capturedVideoUrl = e.detail.url;
});

async function extractVideoUrl() {
  // Ưu tiên URL captured từ fetch interception
  if (capturedVideoUrl) {
    const url = capturedVideoUrl;
    capturedVideoUrl = null;
    return url;
```

```
  }

  // Fallback: lấy từ video element src
  const video = document.querySelector('video[src]');
  if (video?.src && !video.src.startsWith('blob:')) {
    return video.src;
  }

  // Fallback 2: tìm download button và extract href
  const dlBtn = document.querySelector('a[download], [aria-label*="Download"]');
  if (dlBtn?.href) return dlBtn.href;

  throw new Error('Could not extract video URL');
}

// Inject ngay khi content script chạy
injectPageScript();
```

VII. 🖥️ SIDE PANEL UI — Phân Tích Từ Screenshot Thật

```
graph TB
    subgraph "Side Panel Layout (từ screenshot thật)"
        H["🔝 HEADER\nTitle: VEO Automation - Auto Veo on Google Flow\nAuthor: kylenguyen.me ↗\n[User Guide] [Language: English ▾]"]

        TABS["📑 TAB BAR\n⚡ Control | ⚙️ Setting"]

        subgraph "Control Tab"
            R1["Row 1 (Video Modes):\n🎬 Text to Video | 🎞️ Frame to Video | 🧩 Ingredients to Video"]
            R2["Row 2 (Image Modes):\n🖼️ Text to Image | 🔄 Image to Image | 🔗 Last Image To Image"]
            HINT["💡 Hint text:\nSeparate multiple prompts with a blank line.\nLast image will be processed with each prompt."]
            DUR["⏱️ Duration per Prompt: [8s ▾]"]
            QUEUE["📋 Queue Preview:\n• Prompt 1 — thumbnail — 371 chars — [8s ▾]\n• Prompt 2 — thumbnail — 319 chars — [8s ▾]\n• Prompt 3 — thumbnail — 116 chars — [8s ▾]"]
        end
```

```
        subgraph "Progress Section"
            PROG["📊 Video Generation Progress:\n✅ Job 1 — A cute 3D animated... — 100%\n✅ Job 2 — A magical 3D animated... — 100%\n🔄 Job 3 — A young ballerina... — 26%"]
        end
    end

    H --> TABS --> R1 --> R2 --> HINT --> DUR --> QUEUE --> PROG
```

VIII. 🌐 Google Flow Internal API — Cách Extension Tương Tác

```
sequenceDiagram
    participant CS as 🚀 Content Script
    participant UI as 🌐 Google Flow React UI
    participant API as 🔗 aisandbox-pa.googleapis.com
    participant INT as 🔓 MAIN World Interceptor

    Note over CS,API: ─── Phase 1: Setup & Input ───
    CS->>UI: querySelector('textarea') → setReactInputValue(prompt)
    CS->>UI: querySelector('[aria-label="16:9"]') → reactClick()
    CS->>UI: querySelector('button[Generate]') → reactClick()

    Note over CS,API: ─── Phase 2: Generation (30s - 5min) ───
    UI->>API: POST /v1/projects/{id}/generate\n{prompt, model: "veo-3.1", ratio: "16:9"}
    API-->>UI: {operationId: "op_xxx", state: "PROCESSING"}
    INT-->>CS: CustomEvent('__VEO_STATUS__', {state: "PROCESSING"})

    loop Poll mỗi 5 giây
        UI->>API: GET /v1/operations/op_xxx
        API-->>UI: {state: "PROCESSING", progress: 45}
        INT-->>CS: CustomEvent('__VEO_STATUS__', {progress: 45})
        CS->>CS: sendMessage(PROGRESS_UPDATE, 45%)
    end

    Note over CS,API: ─── Phase 3: Completion & Download ───
    API-->>UI: {state: "COMPLETED", downloadUrl: "https://...mp4"}
    INT-->>CS: CustomEvent('__VEO_VIDEO_URL__', {url: "https://...mp4"})
```

```
    CS-->>CS: Return {success: true, videoUrl}
    Note over CS: Service Worker calls chrome.downloads.download()
```

IX. 🔑 10 KỸ THUẬT CHUYÊN SÂU QUAN TRỌNG

1. React Native Setter Trick

```
// ❌ KHÔNG HOẠT ĐỘNG (React ignores)
textarea.value = "my prompt";

// ✅ HOẠT ĐỘNG (bypass React controlled component)
Object.getOwnPropertyDescriptor(HTMLTextAreaElement.prototype, 'value')
  .set.call(textarea, "my prompt");
textarea.dispatchEvent(new Event('input', { bubbles: true }));
```

Tại sao? React override `value` setter trên DOM element. Khi set qua native setter → React internal fiber nhận được event → cập nhật state.

2. MutationObserver vs setTimeout

```
// ❌ KHÔNG ĐỦ TIN CẬY — không biết Google Flow render mất bao lâu
await sleep(3000);

// ✅ REACTIVE — phản ứng ngay khi element xuất hiện
await waitForElement('[data-video-ready]'); // dùng MutationObserver bên trong
```

3. Dual-World Architecture (Isolated + MAIN)

```
ISOLATED WORLD (Content Script):
├── Đọc/ghi DOM ✅
├── Dùng chrome.* APIs ✅
├── Access window.fetch gốc ❌ (chỉ thấy bản copy)
└── Đọc JS variables của page ❌

MAIN WORLD (Injected Script):
├── Override window.fetch ✅
```

```
├── Access page's React internals ✅
├── Dùng chrome.* APIs ❌
└── Giao tiếp qua CustomEvent ↔️ Isolated World
```

4. Selector Resilience (Chống thay đổi DOM)

```
// Ưu tiên cao → thấp:
'[aria-label="Generate"]'              // ✅ Semantic, stable
'[data-testid="generate-btn"]'         // ✅ Testing attr, khá stable
'button[role="button"]'                // ⚠️ Quá chung
'.css-1a2b3c'                          // ❌ Hash thay đổi mỗi deploy
```

5. Auto-Chaining (Last Frame → Next Prompt)

Kỹ thuật "chaining" cho phép tạo chuỗi video liên tục:

- Video 1 hoàn thành → extract frame cuối cùng
- Frame cuối trở thành reference image cho Video 2
- → Tạo sự nhất quán nhân vật/cảnh giữa các video

6. Service Worker Persistence

```
// SW bị Chrome terminate sau 30s không hoạt động!
// Giải pháp: Lưu state vào chrome.storage
await chrome.storage.local.set({ queue: [...] });
// Khi SW restart → restore state và tiếp tục xử lý
```

7. Concurrency Control (Semaphore Pattern)

```
// Chạy TỐI ĐA 2 jobs song song — tránh quá tải Google Flow
if (this.running.size >= this.maxConcurrent) return;
// Queue FIFO: First-In First-Out
```

8. File Upload Automation

```
// Simulate file upload bằng DataTransfer API
const dt = new DataTransfer();
```

```
const file = await urlToFile(imageUrl, 'reference.png');
dt.items.add(file);
const fileInput = document.querySelector('input[type="file"]');
fileInput.files = dt.files;
fileInput.dispatchEvent(new Event('change', { bubbles: true }));
```

9. Multi-Resolution Download

```
// Google Flow cung cấp nhiều quality options
// Extension detect available resolutions và chọn theo user setting
const qualities = ['4k', '2k', '1080p', '720p'];
for (const q of qualities) {
  const btn = document.querySelector(`[data-quality="${q}"]`);
  if (btn) { reactClick(btn); break; }
}
```

10. i18n Dynamic Loading

```
// Đổi ngôn ngữ runtime qua chrome.i18n
// Mỗi string UI dùng message key
document.querySelector('#runBtn').textContent = chrome.i18n.getMessage('startBtn');
// _locales/vi/messages.json: { "startBtn": { "message": "Bắt đầu" } }
```

X. 📦 CÁCH BUILD & PUBLISH LÊN CHROME WEB STORE

```
graph LR
    subgraph "Development"
        A[Viết code\nJS, HTML, CSS] --> B[npm run build\nWebpack/Vite bundle]
        B --> C[Tạo ZIP file\nchứa manifest.json\n+ tất cả assets]
    end

    subgraph "Submission"
        C --> D[Upload ZIP\nlên Chrome\nDeveloper Dashboard]
        D --> E[Điền thông tin:\n• Description\n• Screenshots\n• Privacy policy\n• Category]
```

```
        E --> F[Submit for Review]
    end

    subgraph "Google Review (1-7 ngày)"
        F --> G{Auto scan +\nHuman review}
        G -->|Pass| H[✅ Published!]
        G -->|Fail| I[❌ Rejection\nFix & resubmit]
    end
```

Phí: $5 một lần (developer registration)
Review time: 1-7 ngày (trung bình 2-3 ngày)

XI. 💰 MÔ HÌNH KINH DOANH (Suy luận)

Dựa trên portfolio 4 extensions + Discord community:

- **Free tier:** Sử dụng cơ bản, giới hạn số prompts
- **Premium/Donation:** Unlock tính năng nâng cao (4K, concurrency cao hơn, priority support)
- **Discord community:** Xây dựng cộng đồng người dùng → support + upsell

---

> *[!TIP] **Nếu bạn muốn tự build extension tương tự**, thứ tự học:*
>
> 1. *Chrome Extension Getting Started*
> 2. *Content Scripts + MutationObserver*
> 3. *Service Worker + Message Passing*
> 4. *Side Panel API*
> 5. *React DOM hacking (native setter trick)*
> 6. *Fetch interception (MAIN world injection)*